

Chapter 1

Utility Modules Documentation

1.1 Introduction

This document describes the utility modules used throughout the CTCC: `yaml_loaders.py` (or `smart_loaders`, which dispatches to YAML/JSON by mode), `financial_utils.py`, `calculation_utils.py`, `constants.py`, and `weighted_miles.py`. They provide shared functionality for loading configuration data, financial calculations (PV, AFUDC, discount rate from config), transmission line parameters (phase current, line/converter losses), constants (e.g. `MIN_DISCOUNT_RATE`, `HOURS_PER_YEAR`), and terrain-adjusted distances.

1.2 `yaml_loaders.py`

1.2.1 Method Overview

The `yaml_loaders.py` module provides centralized functions for loading YAML configuration files. It eliminates code duplication by providing a single interface for accessing project parameters, financing details, physical details, and cost parameters.

1.2.2 Key Functions

`load_project_technical_details()`

Loads project technical details from `01_project_technical_details.yaml`.

Returns:

- Tuple: (`construction_type`, `ac_dc`, `capacity_mw`, `conductor_type`, `converter_type`, `line_utilization`, `reconductoring`, `delay_years`, `construction_years`, `project_lifetime`)

`load_physical_details()`

Loads physical project details and returns total miles only.

Returns:

- `total_miles` (float): Sum of all terrain miles

`load_physical_details_detailed()`

Loads physical project details with detailed terrain breakdown.

Returns:

- Tuple: (total_miles, forested_miles, scrubbed_flat_miles, wetland_miles, farmland_miles, desert_barren_miles, urban_miles, rolling_hills_miles, mountain_miles, subsea_miles)

`load_financing_details()`

Loads financing parameters and calculates real WACC using Fisher equation.

Returns:

- Tuple: (inflation_rate, base_year, wacc_nominal, wacc_real)

Key Logic:

```
1 wacc_real = (1 + wacc_nominal) / (1 + inflation_rate) - 1
```

`load_circuit_and_resistance_details(category)`

Loads circuit and resistance details for specified project category.

Parameters:

- `category` (str): Project category identifier

Returns:

- Tuple: (voltage_kv, conductors_per_phase, number_of_phases, number_of_circuits_poles, AC_75_resistance, DC_20_resistance)

Other Loader Functions

Additional loader functions include:

- `load_row_widths(category)` - ROW width for project category
- `load_row_details()` - ROW cost parameters
- `load_delay_costs()` - Delay cost components
- `load_emissions_details()` - Emissions parameters
- `load_congestion_reductions()` - Congestion reduction parameters
- `load_curtailment_reductions()` - Curtailment reduction parameters
- `load_contingencies()` - Contingency percentages
- `load_environmental_mitigation()` - Environmental mitigation parameters

- `load_cost_timing_patterns()` - AFUDC timing patterns
- `load_afudc_config()` - AFUDC configuration
- `load_insurance_details()` - Insurance parameters
- `load_wildfire_costs()` - Wildfire parameters
- `load_outage_costs()` - Outage parameters

1.2.3 Example

```

1 from yaml_loaders import (
2     load_project_technical_details,
3     load_financing_details,
4     load_physical_details
5 )
6
7 # Load project specs
8 construction_type, ac_dc, capacity_mw, ... =
9     load_project_technical_details()
10
11 # Load financing
12 inflation_rate, base_year, wacc_nominal, wacc_real =
13     load_financing_details()
14
15 # Load physical details
16 total_miles = load_physical_details()

```

1.3 financial_utils.py

1.3.1 Method Overview

The `financial_utils.py` module provides shared financial calculation functions: present value (level and growing annuity), COD year, discount rate from config, amortization, and AFUDC capitalization.

Nominal WACC (`r_wacc,nominal`): When capital structure is present and valid (equity_percent, debt_percent, cost_of_equity, cost_of_debt; percentages sum to 1.0; not both costs zero), $r_wacc,nominal = equity_percent \times cost_of_equity + debt_percent \times cost_of_debt$. Otherwise $r_wacc,nominal = financial.wacc_nominal$. This value is used for both discounting (via real WACC from the Fisher equation) and for the AFUDC rate. The helper `get_wacc_nominal(financing_yaml)` returns this value.

1.3.2 Key Functions

`get_discount_rate_from_config(config_yaml, financing_yaml, rate_key="discount_rate_type")`

Returns discount rate and source description from config (e.g. wildfire, outage). Supports `discount_rate_type` or `discount_rate_source`; uses financing YAML for social or WACC real. Returns (`discount_rate`, `source_description`).

`calculate_cod_year(delay_years, construction_years)`

Returns Commercial Operation Date year: `delay_years + construction_years + 1.0` (can be fractional). Used as `start_year` in PV calculations so benefits/costs start at COD.

`calculate_growing_annuity_pv(annual_amount, growth_rate, discount_rate, project_lifetime, delay_years=0, construction_years=0)`

Present value of a growing annuity starting at COD; used for wildfire, outage, and similar streams.

`calculate_nominal_growing_series(annual_amount, growth_rate, project_lifetime)`

Nominal (undiscounted) sum of growing annual amounts over project lifetime.

`validate_discount_rate(discount_rate)`

Raises `ValueError` if discount rate $\leq$ `constants.MIN_DISCOUNT_RATE` (avoids division by zero in PV).

`calculate_present_value(annual_cost, wacc_real, total_years, start_year=1)`

Calculates the present value of annual payments over a given time period.

Parameters:

- `annual_cost` (float): Annual cost amount
- `wacc_real` (float): Real WACC (discount rate)
- `total_years` (float): Number of years
- `start_year` (int): Year when payments begin (default: 1)

Returns:

- `present_value` (float): Present value of the payment stream

Formula:

$$PV = \sum_{t=start}^{start+total-1} \frac{C}{(1+r)^t}$$

`calculate_amortized_cost(principal, wacc_real, project_lifetime)`

Calculates annual amortized cost using standard amortization formula.

Parameters:

- `principal` (float): Initial cost
- `wacc_real` (float): Real WACC (discount rate)
- `project_lifetime` (int): Number of years to amortize

Returns:

- `annual_payment` (float): Annual amortized payment

Formula:

$$A = P \times \frac{r(1+r)^n}{(1+r)^n - 1}$$

where P is principal, r is rate, and n is number of years.

`get_wacc_nominal(financing_yaml)`

Returns nominal WACC (`r_wacc,nominal`). Calls `calculate_afudc_rate(financing_yaml)` and returns the rate; same logic so discounting and AFUDC use the same nominal rate when capital structure is present.

Returns: float — Nominal WACC.

`calculate_afudc_rate(financing_yaml)`

Calculates AFUDC rate from capital structure or falls back to WACC.

Parameters:

- `financing_yaml` (dict): Loaded financing YAML data

Returns:

- Tuple: (`afudc_rate`, `source_description`)

Key Logic:

```
1 if capital_structure specified:
2     rate = equity_percent * cost_of_equity + debt_percent *
      cost_of_debt
3     source = "capital structure"
4 else:
5     rate = wacc_nominal
6     source = "WACC nominal"
```

`calculate_afudc_capitalized_cost(nominal_cost, timing_pattern, delay_years, construction_years, afudc_rate, delay_has_active_work)`

Capitalizes a cost using AFUDC (compounds forward to Commercial Operation Date). If `timing_pattern["afudc_eligible"]` is False, returns `(nominal_cost, 0.0)` with no capitalization.

Parameters:

- `nominal_cost` (float): Total nominal cost
- `timing_pattern` (dict): Dict with `during_delay`, `during_construction`, `afudc_eligible`. When `afudc_eligible` is true, `during_delay` and `during_construction` must sum to 1.0 (within tolerance 0.001) so that 100% of the cost is allocated to delay and/or construction.
- `delay_years` (float): Number of delay years
- `construction_years` (float): Number of construction years
- `afudc_rate` (float): AFUDC rate (annual)
- `delay_has_active_work` (bool): Whether active CWIP work continues during delay

Returns:

- Tuple: `(capitalized_cost, afudc_amount)`

Key Logic:

```
1 # Cost during delay period (only if afudc_eligible)
2 delay_cost = nominal_cost * timing_pattern["during_delay"]
3 if delay_cost > 0:
4     if delay_has_active_work and delay_years > 0:
5         # AFUDC during delay: uniform spending, compound from
6         # midpoint to COD
7         avg_years_to_cod = total_years_to_cod - (delay_years / 2)
8         capitalized_cost += delay_cost * (1 + afudc_rate) **
9         avg_years_to_cod
10    else:
11        if delay_years > 0:
12            # No active work: AFUDC suspended during delay, compound
13            # only during construction
14            capitalized_cost += delay_cost * (1 + afudc_rate) **
15            construction_years
16        else:
17            capitalized_cost += delay_cost # No delay period
18
19 # Cost during construction period
20 construction_cost = nominal_cost * timing_pattern["
    during_construction"]
```

```

17 if construction_cost > 0 and construction_years > 0:
18     # Uniform spending: compound from midpoint of construction to
      COD
19     avg_years_to_cod = construction_years / 2
20     capitalized_cost += construction_cost * (1 + afudc_rate) **
      avg_years_to_cod
21 else:
22     capitalized_cost += construction_cost
23
24 afudc_amount = capitalized_cost - nominal_cost

```

1.3.3 Example

```

1 from financial_utils import (
2     calculate_present_value,
3     calculate_amortized_cost,
4     calculate_afudc_capitalized_cost
5 )
6
7 # Present value of annual O&M costs
8 pv = calculate_present_value(
9     annual_cost=1000000,
10    wacc_real=0.03,
11    total_years=50,
12    start_year=3
13 )
14
15 # Amortize build cost
16 annual_payment = calculate_amortized_cost(
17     principal=100000000,
18     wacc_real=0.03,
19     project_lifetime=50
20 )
21
22 # AFUDC capitalization
23 capitalized, afudc = calculate_afudc_capitalized_cost(
24     nominal_cost=100000000,
25     timing_pattern={"during_delay": 0.1, "during_construction": 0.9,
26                     "afudc_eligible": True},
27     delay_years=2,
28     construction_years=2,
29     afudc_rate=0.05,
30     delay_has_active_work=False

```

1.4 calculation_utils.py

1.4.1 Method Overview

The `calculation_utils.py` module provides shared calculation functions for transmission line parameters, including phase current, full load adjustment, line losses, and converter losses.

1.4.2 Key Functions

`calculate_phase_current(capacity_mw, voltage_kv, number_of_phases, number_of_circuits_poles, ac_dc)`

Calculates phase current based on AC or DC transmission type.

Parameters:

- `capacity_mw` (float or str): Line capacity in MW
- `voltage_kv` (float): Voltage in kV
- `number_of_phases` (int): Number of phases
- `number_of_circuits_poles` (int): Number of circuits/poles
- `ac_dc` (str): "AC" or "DC"

Returns:

- `phase_current` (float): Phase current in Amps

Formulas:

$$I_{AC} = \frac{P \times 1000}{0.95 \times V \times \sqrt{N_{phases}} \times N_{circuits}} \quad (1.1)$$

$$I_{DC} = \frac{P \times 1000}{V \times \sqrt{N_{phases}} \times N_{poles}} \quad (1.2)$$

`full_load_adjusted(line_utilization_percent)`

Calculates full load adjustment based on line utilization.

Parameters:

- `line_utilization_percent` (float): Line utilization as decimal (0-1)

Returns:

- `adjustment_factor` (float): Full load adjustment factor

Formula:

$$\text{Adjustment} = \frac{u + u^2}{2}$$

where u is the utilization factor.

`calculate_line_losses(...)`

Calculates line losses for transmission lines.

Parameters:

- `phase_current` (float): Phase current in Amps
- `full_load_adj` (float): Full load adjustment factor
- `AC_75_resistance` (float): AC resistance at 75°C (ohms/mile)
- `DC_20_resistance` (float): DC resistance at 20°C (ohms/mile)
- `ac_dc` (str): "AC" or "DC"
- `number_of_circuits_poles` (int): Number of circuits/poles
- `conductors_per_phase` (int): Number of conductors per phase
- `number_of_phases` (int): Number of phases
- `line_length` (float): Line length in miles
- `project_lifetime` (int): Project lifetime in years
- `capacity_mw_numeric` (float): Capacity in MW
- `line_utilization_percent` (float): Line utilization as decimal

Returns:

- Tuple: (losses_mwh_per_year, lifetime_losses_mwh, losses_mw_per_mile, resistance_per_mile, line_loss_per_mile_percent, total_line_loss_mw, total_line_loss_percent)

Key Logic:

```
1 # Select resistance based on AC/DC
2 resistance_per_mile = AC_75_resistance if ac_dc == "AC" else
   DC_20_resistance
3
4 # Calculate losses per mile
5 number_of_conductors = conductors_per_phase * number_of_phases *
   number_of_circuits_poles
6 losses_mw_per_mile = ((phase_current / conductors_per_phase) ** 2
7                       * number_of_conductors
8                       * resistance_per_mile
9                       * full_load_adj / 1000000)
10
11 # Total losses
12 total_line_loss_mw = losses_mw_per_mile * line_length
13 losses_mwh_per_year = total_line_loss_mw * 8760
14 lifetime_losses_mwh = losses_mwh_per_year * project_lifetime
```

`calculate_converter_losses(number_of_converters, converter_type, line_utilization_percent, capacity_mw_numeric, ac_dc, converter_loss_percentage=None)`

Calculates converter losses based on number of converters (DC only). Uses `converter_loss_percentage` from project if provided; otherwise `constants.LCC_CONVERTER_LOSS` or `VSC_CONVERTER_LOSS` by converter type.

Parameters:

- `number_of_converters` (int): Number of converters
- `converter_type` (str): Converter type (LCC or VSC)
- `line_utilization_percent` (float): Line utilization as decimal
- `capacity_mw_numeric` (float): Capacity in MW
- `ac_dc` (str): "AC" or "DC"
- `converter_loss_percentage` (float, optional): Override loss rate from project

Returns:

- Tuple: (total_converter_losses_mw, total_converter_losses_mwh, converter_loss_percent)

Key Logic:

```
1 if ac_dc == "DC":
2     converter_loss_percentage = 0.0075 if "LCC" in converter_type
3     else 0.01
4     converter_losses_mw = (converter_loss_percentage
5                           * line_utilization_percent
6                           * capacity_mw_numeric)
7     total_converter_losses_mw = converter_losses_mw *
8     number_of_converters
9     total_converter_losses_mwh = total_converter_losses_mw * 8760
10 else:
11     total_converter_losses_mw = 0
12     total_converter_losses_mwh = 0
```

Other calculation_utils functions

- `build_category_string(...)` — Build project category for YAML/JSON lookups (e.g. Overhead/AC/500MW/Advanced Aluminum/NA).
- `normalize_construction_type_for_yaml(construction_type)` — Map script construction type to YAML key (e.g. overhead → Overhead).
- `get_converter_type(ac_dc, converter_type)` — Return converter type for category (e.g. AC → NA).

- `miles_to_acres(miles, row_width_feet)` — Convert ROW miles to acres using `constants.FEET_PER_MILE` and `SQUARE_FEET_PER_ACRE`.
- `to_percent(decimal)`, `from_percent(percentage)` — Convert between decimal and percentage.

1.4.3 Example

```

1 from calculation_utils import (
2     calculate_phase_current,
3     full_load_adjusted,
4     calculate_line_losses,
5     build_category_string,
6 )
7
8 # Calculate phase current
9 phase_current = calculate_phase_current(
10     capacity_mw=500,
11     voltage_kv=345,
12     number_of_phases=3,
13     number_of_circuits_poles=1,
14     ac_dc="AC"
15 )
16
17 # Calculate full load adjustment
18 full_load_adj = full_load_adjusted(0.7) # 70% utilization
19
20 # Calculate line losses
21 losses_mwh_per_year, lifetime_losses_mwh, ... =
22     calculate_line_losses(
23         phase_current, full_load_adj, AC_75_resistance, DC_20_resistance
24         ,
25         "AC", 1, 2, 3, 100, 50, 500, 0.7
26 )

```

1.5 weighted_miles.py

1.5.1 Method Overview

The `weighted_miles.py` module calculates weighted miles and average terrain multiplier for transmission lines. Weighted miles account for terrain complexity by multiplying miles in each terrain type by a terrain-specific cost multiplier.

1.5.2 Key Functions

`calculate_weighted_miles()`

Calculates weighted miles and average terrain multiplier from project physical details.

Returns:

- Tuple: (weighted_miles, average_terrain_multiplier)

Calculation:

$$\text{Weighted Miles} = \sum_{\text{terrains}} \text{Miles}_{\text{terrain}} \times \text{Multiplier}_{\text{terrain}} \quad (1.3)$$

$$\text{Average Multiplier} = \frac{\text{Weighted Miles}}{\text{Total Miles}} \quad (1.4)$$

Key Logic:

```
1 weighted_miles = 0
2 for terrain_type, terrain_miles in terrain_miles_dict.items():
3     multiplier = terrain_multipliers[terrain_type]
4     weighted_miles += terrain_miles * multiplier
5
6 average_terrain_multiplier = weighted_miles / total_miles
```

1.5.3 Example

```
1 from weighted_miles import calculate_weighted_miles
2
3 # Calculate weighted miles
4 weighted_miles, avg_multiplier = calculate_weighted_miles()
5
6 # Example calculation:
7 # Terrain distribution:
8 #   - Forested: 40 miles, multiplier: 2.25
9 #   - Farmland: 60 miles, multiplier: 1.0
10 #   - Total: 100 miles
11 #
12 # Weighted miles = (40 * 2.25) + (60 * 1.0) = 150 miles
13 # Average multiplier = 150 / 100 = 1.5
```

1.6 Integration Examples

1.6.1 Combined Usage

```

1 # Typical usage in a cost calculation script:
2 from yaml_loaders import (
3     load_project_technical_details,
4     load_financing_details,
5     load_physical_details
6 )
7 from financial_utils import calculate_present_value
8 from weighted_miles import calculate_weighted_miles
9
10 # Load configuration
11 construction_type, ac_dc, capacity_mw, ... =
12     load_project_technical_details()
13 inflation_rate, base_year, wacc_nominal, wacc_real =
14     load_financing_details()
15 total_miles = load_physical_details()
16
17 # Calculate weighted miles
18 weighted_miles, avg_multiplier = calculate_weighted_miles()
19
20 # Calculate present value
21 pv = calculate_present_value(
22     annual_cost=1000000,
23     wacc_real=wacc_real,
24     total_years=50,
25     start_year=3
26 )

```

1.7 Key Implementation Details

1.7.1 Path Resolution

All YAML loaders use relative paths ("../yaml/") assuming scripts are run from the project root directory. The `csv_output_manager.py` uses dynamic path resolution to handle different execution contexts.

1.7.2 Fisher Equation

The real WACC calculation uses the Fisher equation to convert nominal WACC to real WACC, accounting for inflation:

$$r_{real} = \frac{1 + r_{nominal}}{1 + \pi} - 1$$

1.7.3 Full Load Adjustment

The full load adjustment accounts for the fact that losses are proportional to current squared, and average current is less than peak current. The formula $(u + u^2)/2$ provides a better approximation than simple u^2 for typical utilization patterns.

1.7.4 AFUDC Timing

AFUDC capitalization compounds costs forward to Commercial Operation Date (COD). The calculation accounts for:

- Costs during delay period (may or may not have AFUDC depending on active work)
- Costs during construction period (always has AFUDC)
- Uniform spending assumption (uses midpoint of each period)

1.7.5 Terrain Multipliers

Terrain multipliers reflect the relative cost/complexity of construction in different terrain types. For example, forested terrain may have a multiplier of 2.25, meaning construction costs 2.25x more per mile than baseline terrain.

1.8 constants.py

1.8.1 Method Overview

The `constants.py` module centralizes magic numbers used across CTCC scripts.

1.8.2 Key Constants

- `FEET_PER_MILE`, `SQUARE_FEET_PER_ACRE`, `HOURS_PER_YEAR`
- `MIN_DISCOUNT_RATE` — Minimum allowed discount rate (e.g. -0.99) to avoid division by zero in PV
- `GROWTH_RATE_TOLERANCE`, `DISCOUNT_GROWTH_EQUALITY_TOLERANCE`
- `LCC_CONVERTER_LOSS`, `VSC_CONVERTER_LOSS` — Converter loss fractions
- `AC_POWER_FACTOR` — Power factor for AC phase current
- `CONSTRUCTION_TYPE_OVERHEAD`, `CONSTRUCTION_TYPE_UNDERGROUND`, `CONSTRUCTION_TYPE_SUBSEA`, etc.
- `CONVERTER_TYPE_NA` — Used when AC (no converter)